



CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU
CURSO DE CIÊNCIA DA COMPUTAÇÃO

GUIA LOCAL INTELIGENTE

Análise End-to-End da Arquitetura, Frontend, Backend, Banco
de Dados e Integrações Externas

Integrantes

Ruan Carlos

Kauã Nylo

Lucas Silva

Bernardo Franco

Disciplina: Códigos de Alta Performance Web

Professor(a): Anderlan Henrique

Maceió - AL

2026

GUIA LOCAL INTELIGENTE

Trabalho apresentado ao curso de Ciência da Computação do Centro Universitário Maurício de Nassau — UNINASSAU, como requisito parcial para avaliação acadêmica, contendo análise técnica end-to-end da aplicação Guia Local Inteligente, composta por frontend web, backend REST, autenticação, banco de dados e integração com APIs externas.

Professor(a): Anderlan Henrique

Resumo

Este trabalho apresenta uma análise end-to-end da aplicação **Guia Local Inteligente**, projeto acadêmico desenvolvido para auxiliar usuários na consulta de informações regionais a partir de um CEP. A solução é dividida em dois repositórios principais: um backend em Node.js, Express, TypeScript, Prisma, PostgreSQL e BetterAuth; e um frontend web em React, TypeScript, Vite, Tailwind CSS e React Router. O sistema permite autenticação social com Google, busca de endereço por CEP, consulta de coordenadas geográficas, obtenção de dados climáticos, listagem de estabelecimentos próximos e gerenciamento de favoritos por usuário autenticado. A análise contempla arquitetura geral, fluxo de dados, camadas do sistema, requisitos funcionais e não funcionais, banco de dados, autenticação, integrações externas, telas, componentes, segurança, implantação e pontos de melhoria.

Palavras-chave: Guia Local Inteligente. React. Vite. Express. Prisma. PostgreSQL. BetterAuth. API REST. APIs externas.

Abstract

This paper presents an end-to-end analysis of the **Guia Local Inteligente** application, an academic project designed to help users retrieve regional information based on a Brazilian ZIP code. The solution is divided into two main repositories: a backend built with Node.js, Express, TypeScript, Prisma, PostgreSQL and BetterAuth; and a web frontend built with React, TypeScript, Vite, Tailwind CSS and React Router. The system supports Google social authentication, address lookup by ZIP code, geocoding, weather data retrieval, nearby place discovery and authenticated favorite place management. This document covers the general architecture, data flow, system layers, functional and non-functional requirements, database, authentication, external integrations, user interface, security, deployment and improvement opportunities.

Keywords: Local Guide. React. Vite. Express. Prisma. PostgreSQL. BetterAuth. REST API. External APIs.

Sumário

1	INTRODUÇÃO	6
1.1	Contextualização	6
1.2	Problema	6
1.3	Justificativa	6
2	OBJETIVOS	7
2.1	Objetivo Geral	7
2.2	Objetivos Específicos	7
3	VISÃO GERAL DO SISTEMA	8
3.1	Descrição do Produto	8
3.2	Repositórios Analisados	8
3.3	Principais Funcionalidades	8
4	ARQUITETURA DA APLICAÇÃO	9
4.1	Estilo Arquitetural	9
4.2	Arquitetura em Camadas	9
4.3	Diagrama Geral da Arquitetura	10

5	TECNOLOGIAS UTILIZADAS	11
5.1	Frontend	11
5.2	Backend	11
6	ESTRUTURA DOS PROJETOS	13
6.1	Estrutura do Backend	13
6.2	Estrutura do Frontend	13
7	LEVANTAMENTO DE REQUISITOS	15
7.1	Requisitos Funcionais	15
7.2	Requisitos Não Funcionais	15
8	ANÁLISE DO BACKEND	17
8.1	Visão Geral	17
8.2	Scripts do Backend	17
8.3	Configuração da Aplicação Express	17
8.4	CORS e Credenciais	18
8.5	Health Check	18
8.6	Rota de Usuário Atual	18
9	AUTENTICAÇÃO E SESSÃO	19
9.1	BetterAuth no Backend	19
9.2	BetterAuth no Frontend	20
9.3	Fluxo de Login com Google	20
9.4	Middleware de Proteção de Rotas	20
10	BANCO DE DADOS E MODELO RELACIONAL	22
10.1	Prisma e PostgreSQL	22
10.2	Entidades Principais	22
10.3	Modelo FavoritePlace	22
10.4	Relacionamentos	23
10.5	Diagrama Entidade-Relacionamento Simplificado	23
11	FLUXO DE BUSCA POR CEP	24
11.1	Visão Geral do Fluxo	24
11.2	Fluxo Ponta a Ponta	24
11.3	Orquestração no Service	24
11.4	Validação do CEP	25
11.5	Integração com ViaCEP	25
11.6	Integração com Open-Meteo Geocoding	25
11.7	Integração com Open-Meteo Forecast	26

11.8	Integração com Overpass API	26
12	SISTEMA DE FAVORITOS	27
12.1	Visão Geral	27
12.2	Rotas de Favoritos	27
12.3	Adicionar Favorito	27
12.4	Listar Favoritos	28
12.5	Remover Favorito	28
12.6	FavoritesProvider no Frontend	28
13	ANÁLISE DO FRONTEND	29
13.1	Visão Geral	29
13.2	Scripts do Frontend	29
13.3	Rotas da Aplicação	29
13.4	Cliente HTTP	29
13.5	Hook useSearch	30
13.6	Providers	30
13.7	Páginas Principais	30
13.7.1	Tela de Login	30
13.7.2	Tela Inicial	31
13.7.3	Tela de Favoritos	31
13.7.4	Tela Sobre	31
14	FLUXOS END-TO-END	32
14.1	Fluxo 1: Login do Usuário	32
14.2	Fluxo 2: Busca por CEP	32
14.3	Fluxo 3: Adicionar Favorito	32
14.4	Fluxo 4: Remover Favorito	33
15	APIS EXTERNAS	34
15.1	ViaCEP	34
15.2	Open-Meteo Geocoding	34
15.3	Open-Meteo Forecast	34
15.4	Overpass API / OpenStreetMap	34
15.5	Análise das Integrações	34
16	DEPLOY E CONFIGURAÇÃO DE AMBIENTE	35
16.1	Variáveis do Backend	35
16.2	Variáveis do Frontend	35
16.3	Pipeline de Build	35
16.3.1	Backend	35

16.3.2	Frontend	35
17	TESTES RECOMENDADOS	37
17.1	Testes Funcionais	37
17.2	Testes de Integração	37
17.3	Testes de Usabilidade	37
18	PONTOS FORTES, LIMITAÇÕES E MELHORIAS	38
18.1	Pontos Fortes	38
18.2	Limitações Identificadas	38
18.3	Melhorias Futuras	39
19	DIFICULDADES ENCONTRADAS	40
19.1	Autenticação em Ambientes Diferentes	40
19.2	CORS e Cookies	40
19.3	APIs Externas	40
19.4	Ambiguidade Geográfica	40
20	RESULTADOS E DISCUSSÃO	41
21	CONCLUSÃO	42
22	TRABALHOS FUTUROS	43
23	REPOSITÓRIOS DO PROJETO	44
	REFERÊNCIAS	45
A	APÊNDICE A – RESUMO TÉCNICO PARA APRESENTAÇÃO .	46
A.1	Resumo do Backend	46
A.2	Resumo do Frontend	46
A.3	Resumo do Fluxo Principal	46
B	APÊNDICE B – CHECKLIST DE DEFESA ORAL	47

1 Introdução

1.1 Contextualização

A aplicação **Guia Local Inteligente** surge em um cenário no qual usuários precisam acessar informações rápidas sobre uma região, como endereço, clima e locais próximos. Em vez de exigir que o usuário pesquise manualmente em diferentes plataformas, a aplicação centraliza essas consultas em uma interface única. A entrada principal do sistema é o CEP informado pelo usuário. A partir dele, o backend busca o endereço, obtém coordenadas geográficas, consulta dados meteorológicos e pesquisa estabelecimentos próximos.

O projeto é dividido em dois repositórios:

- **Guia_Local_Inteligente-APIREST**: backend responsável por autenticação, regras de negócio, integração com banco de dados e comunicação com APIs externas.
- **Guia_Local_Inteligente-Client**: frontend web responsável pela experiência visual, navegação, telas, chamadas HTTP e interação do usuário com o sistema.

1.2 Problema

Usuários frequentemente precisam consultar várias informações sobre uma região: endereço, cidade, clima atual, temperatura mínima e máxima, locais próximos e pontos de interesse. Normalmente, essas informações ficam espalhadas em serviços diferentes. O problema tratado pelo projeto é: **como centralizar informações úteis de uma região a partir de um CEP, com autenticação, favoritos e interface simples?**

1.3 Justificativa

O projeto se justifica por unir conceitos importantes de desenvolvimento moderno: arquitetura cliente-servidor, API REST, banco relacional, autenticação social, consumo de APIs públicas, tratamento de erros, responsividade e separação de responsabilidades. Além de ser funcional, o projeto demonstra práticas comuns no mercado, como frontend desacoplado do backend, deploy separado, uso de variáveis de ambiente e persistência em PostgreSQL.

2 Objetivos

2.1 Objetivo Geral

Analisar detalhadamente a aplicação Guia Local Inteligente de ponta a ponta, descrevendo sua arquitetura, tecnologias, fluxos, camadas, banco de dados, autenticação, integrações externas, frontend, backend e oportunidades de evolução.

2.2 Objetivos Específicos

- Identificar as tecnologias utilizadas no frontend e no backend;
- Descrever a arquitetura cliente-servidor da aplicação;
- Explicar o fluxo completo de autenticação com Google via BetterAuth;
- Detalhar o fluxo de busca por CEP e integração com APIs externas;
- Analisar o funcionamento do sistema de favoritos;
- Mapear as entidades do banco de dados;
- Documentar requisitos funcionais e não funcionais;
- Apontar pontos fortes, limitações e melhorias futuras.

3 Visão Geral do Sistema

3.1 Descrição do Produto

O Guia Local Inteligente é uma aplicação web que permite que um usuário autenticado pesquise uma região por CEP. O sistema retorna informações de endereço, clima e locais próximos. A aplicação também permite salvar estabelecimentos favoritos, vinculando esses locais ao usuário logado.

3.2 Repositórios Analisados

Repositório	Responsabilidade
Guia_Local_Inteligente-Api-REST	Backend da aplicação. Contém servidor Express, configuração do BetterAuth, Prisma, modelos do banco, rotas de busca e rotas de favoritos.
Guia_Local_Inteligente-Client	Frontend web. Contém aplicação React com Vite, telas, componentes visuais, providers, hooks, configuração de autenticação e comunicação HTTP com o backend.

3.3 Principais Funcionalidades

- Login social com Google;
- Controle de sessão por cookies;
- Busca de região por CEP;
- Consulta de endereço por ViaCEP;
- Consulta de coordenadas por Open-Meteo Geocoding;
- Consulta de clima por Open-Meteo Forecast;
- Consulta de locais próximos por Overpass API/OpenStreetMap;
- Exibição de resultados na interface;
- Modal/detalhamento de local;
- Cadastro, listagem e remoção de favoritos;
- Navegação entre início, favoritos, sobre e login.

4 Arquitetura da Aplicação

4.1 Estilo Arquitetural

A aplicação adota uma arquitetura **cliente-servidor**, com separação entre frontend e backend. O frontend é responsável pela interface e pela experiência do usuário. O backend concentra regras de negócio, autenticação, acesso ao banco de dados e chamadas para serviços externos. Essa divisão facilita manutenção, escalabilidade e organização do projeto.

4.2 Arquitetura em Camadas

A solução pode ser compreendida em cinco camadas principais:

1. **Camada de Apresentação:** React, componentes, páginas e estilização.
2. **Camada de Estado e Hooks:** providers, hooks de busca, favoritos e autenticação.
3. **Camada de API Backend:** Express, rotas REST e middlewares.
4. **Camada de Serviços:** regras de negócio, orquestração de APIs externas e operações com favoritos.
5. **Camada de Persistência:** Prisma ORM e PostgreSQL.

4.3 Diagrama Geral da Arquitetura

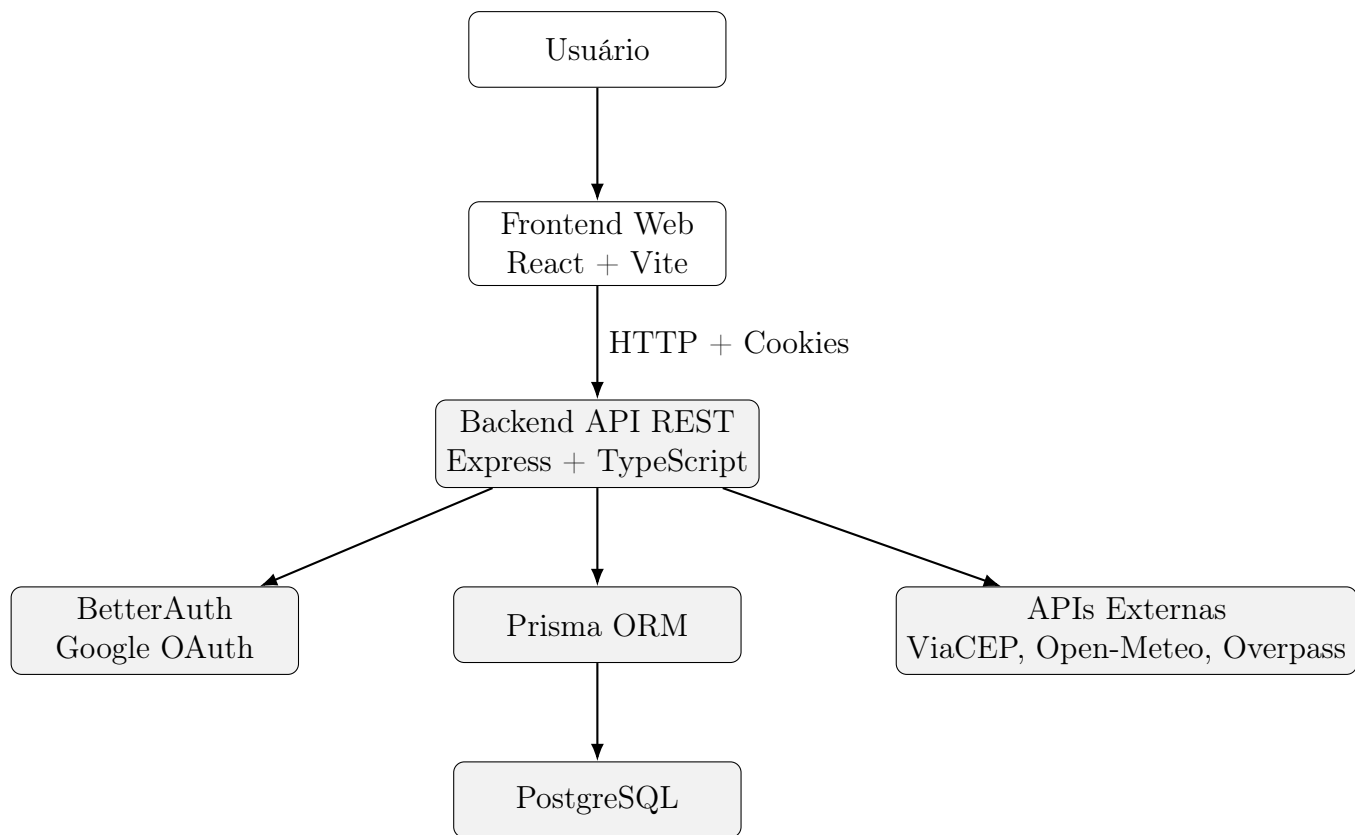


Figura 1 – Arquitetura geral do Guia Local Inteligente

5 Tecnologias Utilizadas

5.1 Frontend

O frontend utiliza tecnologias modernas do ecossistema JavaScript/TypeScript:

Tecnologia	Função no Projeto
React	Construção da interface baseada em componentes reutilizáveis.
TypeScript	Tipagem estática para reduzir erros e melhorar manutenção.
Vite	Ferramenta de build e desenvolvimento rápido.
React Router DOM	Controle de navegação entre páginas como início, login, favoritos e sobre.
Tailwind CSS	Estilização utilitária e responsiva.
Axios	Cliente HTTP configurado para comunicação com a API.
BetterAuth React Client	Cliente de autenticação usado no frontend para sessão, login social e logout.
Lucide React	Ícones utilizados na interface.
Vaul / Base UI / Shadcn	Apoio na construção de componentes interativos, modais e elementos de interface.

5.2 Backend

O backend foi desenvolvido com foco em API REST, autenticação e persistência:

Tecnologia	Função no Projeto
Node.js	Ambiente de execução JavaScript no servidor.
Express	Framework HTTP responsável por rotas, middlewares e respostas da API.
TypeScript	Tipagem e organização do código backend.
BetterAuth	Autenticação com Google, sessão e gerenciamento de usuários.
Prisma ORM	Mapeamento objeto-relacional e acesso ao banco PostgreSQL.

PostgreSQL	Banco de dados relacional para usuários, sessões, contas e favoritos.
CORS	Controle de origem permitida e envio de credenciais entre frontend e backend.
Dotenv	Leitura de variáveis de ambiente.

6 Estrutura dos Projetos

6.1 Estrutura do Backend

A organização do backend segue separação por responsabilidades:

```
1 Guia_Local_Inteligente - ApiREST /
2     prisma /
3         schema.prisma
4     src /
5         @types /
6         controllers /
7             favorite.controller.ts
8             search.controller.ts
9         lib /
10            auth.ts
11            get-session-user.ts
12            prisma.ts
13        routes /
14            favorite.route.ts
15            search.route.ts
16        services /
17            favorite.service.ts
18            search.service.ts
19        app.ts
20        server.ts
21    package.json
22    prisma.config.ts
23    tsconfig.json
```

Listing 6.1 – Estrutura resumida do backend

6.2 Estrutura do Frontend

O frontend é organizado em componentes, páginas, hooks, providers, interfaces e bibliotecas auxiliares:

```
1 Guia_Local_Inteligente - Client /
2     public /
3     src /
4         assets /
5         components /
6         pages /
7             inicio /
```

```
8             favoritos/
9             login/
10            sobre/
11            modals/
12            navigationMobile/
13    hooks/
14            useSearch.ts
15            useFavorites.ts
16    interfaces/
17    lib/
18            axios.ts
19            auth-client.ts
20    providers/
21            AuthProvider.tsx
22            FavoritesProvider.tsx
23    utils/
24    App.tsx
25    index.css
26    main.tsx
27    Dockerfile
28    docker-compose.yml
29    nginx.conf
30    package.json
31    vite.config.ts
```

Listing 6.2 – Estrutura resumida do frontend

7 Levantamento de Requisitos

7.1 Requisitos Funcionais

Código	Descrição
RF01	O sistema deve permitir login social com conta Google.
RF02	O sistema deve manter sessão do usuário por meio do BetterAuth e cookies.
RF03	O sistema deve permitir consulta de região por CEP.
RF04	O sistema deve validar se o CEP possui 8 dígitos numéricos.
RF05	O sistema deve consultar endereço a partir da API ViaCEP.
RF06	O sistema deve obter coordenadas da cidade por meio da API de geocodificação da Open-Meteo.
RF07	O sistema deve consultar clima atual, temperatura mínima e máxima.
RF08	O sistema deve buscar locais próximos usando a Overpass API/OpenStreetMap.
RF09	O sistema deve apresentar os resultados da busca no frontend.
RF10	O sistema deve permitir salvar locais como favoritos.
RF11	O sistema deve listar favoritos do usuário autenticado.
RF12	O sistema deve permitir remover um favorito.
RF13	O sistema deve impedir acesso às rotas de busca e favoritos por usuários não autenticados.
RF14	O sistema deve possuir navegação entre início, favoritos, sobre e login.
RF15	O sistema deve exibir estados de carregamento, sucesso e erro durante a busca.

7.2 Requisitos Não Funcionais

Código	Descrição
RNF01	A aplicação deve ser responsiva, com foco em uso web/mobile.
RNF02	A comunicação entre frontend e backend deve utilizar HTTP/HTTPS.
RNF03	A autenticação deve utilizar cookies seguros em produção.

RNF04	O backend deve validar variáveis de ambiente obrigatórias no momento da inicialização.
RNF05	O banco de dados deve manter integridade referencial entre usuários, sessões, contas e favoritos.
RNF06	O sistema deve tratar erros de APIs externas e retornar mensagens compreensíveis.
RNF07	A aplicação deve manter o frontend desacoplado do backend.
RNF08	O código deve ser organizado em camadas para facilitar manutenção.
RNF09	A API deve permitir CORS apenas para a origem configurada no ambiente.
RNF10	O sistema deve ser preparado para deploy em serviços como Vercel, Render ou infraestrutura similar.

8 Análise do Backend

8.1 Visão Geral

O backend é uma API REST construída com Express e TypeScript. Ele expõe rotas para autenticação, verificação de saúde, consulta de usuário atual, busca por CEP e gerenciamento de favoritos. A aplicação utiliza BetterAuth para autenticação, Prisma para comunicação com PostgreSQL e serviços internos para organizar as regras de negócio.

8.2 Scripts do Backend

O arquivo `package.json` do backend define os comandos principais:

```
1 {  
2   "dev": "tsx watch src/server.ts",  
3   "build": "npm run prisma:generate && tsc",  
4   "start": "node dist/server.js",  
5   "prisma:generate": "prisma generate",  
6   "prisma:deploy": "prisma migrate deploy",  
7   "prisma:push": "prisma db push",  
8   "prisma:studio": "prisma studio"  
9 }
```

Listing 8.1 – Scripts principais do backend

Esses scripts mostram que o backend possui fluxo de desenvolvimento com `tsx watch`, build com geração do Prisma Client e compilação TypeScript, além de comandos específicos para migração e inspeção do banco.

8.3 Configuração da Aplicação Express

O arquivo `src/app.ts` centraliza a configuração do servidor Express. Ele configura CORS, BetterAuth, JSON parser, health check e rotas de domínio:

```
1 const app = express();  
2  
3 app.use(cors({  
4   origin: allowedOrigins,  
5   methods: ["GET", "POST", "PUT", "DELETE"],  
6   credentials: true,  
7 }));  
8  
9 app.all(/\/api\/auth\/.*/, toNodeHandler(auth));
```

```
10 app.use(express.json());
11
12 app.get("/api/health", (req, res) => {
13   res.json({ ok: true });
14 });
15
16 app.use("/api/favorites", favoriteRoutes);
17 app.use("/api/search", searchRoutes);
```

Listing 8.2 – Configuração resumida do Express

A ordem é importante: o handler do BetterAuth é registrado antes do `express.json()`, pois bibliotecas de autenticação podem exigir leitura específica da requisição. Depois são registradas as rotas de negócio.

8.4 CORS e Credenciais

O backend usa a variável `FRONTEND_URL` para montar a lista de origens permitidas. O CORS está configurado com `credentials: true`, permitindo que cookies de sessão sejam enviados entre frontend e backend. Essa configuração é indispensável para login persistente quando frontend e backend estão em domínios diferentes.

8.5 Health Check

A rota `/api/health` retorna `ok: true`. Ela serve para verificar se a API está online, sendo útil em deploys, monitoramento e testes rápidos.

8.6 Rota de Usuário Atual

A rota `/api/me` consulta a sessão atual através do BetterAuth e retorna os dados da sessão. Ela é útil para validar se o usuário está logado e para obter informações do usuário autenticado.

9 Autenticação e Sessão

9.1 BetterAuth no Backend

A autenticação é configurada em `src/lib/auth.ts`. O backend exige as seguintes variáveis de ambiente:

- `DATABASE_URL`
- `BETTER_AUTH_SECRET`
- `BETTER_AUTH_URL`
- `FRONTEND_URL`
- `GOOGLE_CLIENT_ID`
- `GOOGLE_CLIENT_SECRET`

Essa validação em tempo de inicialização é positiva, pois evita que a aplicação suba com configuração incompleta. O BetterAuth é configurado com adaptador Prisma, provedor PostgreSQL e login social com Google.

```
1 export const auth = betterAuth({
2   appName: "GLI",
3   baseURL: process.env.BETTER_AUTH_URL,
4   trustedOrigins: [process.env.FRONTEND_URL],
5   advanced: {
6     defaultCookieAttributes: {
7       sameSite: "none",
8       secure: true,
9     },
10  },
11  database: prismaAdapter(prisma, { provider: "postgresql" }),
12  socialProviders: {
13    google: {
14      clientId: process.env.GOOGLE_CLIENT_ID,
15      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
16    },
17  },
18 });
```

Listing 9.1 – Configuração resumida do BetterAuth

9.2 BetterAuth no Frontend

No frontend, o cliente de autenticação é criado com `createAuthClient`, apontando para a URL da API:

```
1 export const authClient = createAuthClient({  
2   baseURL: import.meta.env.VITE_API_URL || "http://localhost:3000",  
3 });
```

Listing 9.2 – Cliente de autenticação no frontend

O `AuthProvider` encapsula o estado de autenticação, consulta a sessão com `authClient.useSession` e disponibiliza funções de login e logout para a aplicação.

9.3 Fluxo de Login com Google

O fluxo de autenticação ocorre da seguinte forma:

1. O usuário acessa a tela de login.
2. O frontend chama `signInWithGoogle()`.
3. O BetterAuth redireciona o usuário para o provedor Google.
4. Após autorização, o Google retorna para o backend.
5. O BetterAuth cria ou atualiza usuário, conta e sessão no banco.
6. O backend grava cookies de sessão.
7. O usuário retorna ao frontend na URL definida em `callbackURL`.
8. O frontend passa a reconhecer o usuário por `authClient.useSession()`.

9.4 Middleware de Proteção de Rotas

O backend possui um middleware `requireAuth`. Ele consulta a sessão usando os cabeçalhos da requisição e bloqueia usuários não autenticados:

```
1 export async function requireAuth(req, res, next) {  
2   const user = await getSessionUser(req);  
3  
4   if (!user) {  
5     return res.status(401).json({  
6       message: "Usuário não autenticado.",  
7     });  
8   }
```

```
9
10   req.user = user;
11   next();
12 }
```

Listing 9.3 – Middleware requireAuth

Esse middleware é aplicado nas rotas de busca e favoritos, tornando a autenticação obrigatória para essas funcionalidades.

10 Banco de Dados e Modelo Relacional

10.1 Prisma e PostgreSQL

O projeto utiliza Prisma como ORM e PostgreSQL como banco relacional. O `schema.prisma` define modelos relacionados à autenticação e ao domínio de favoritos.

10.2 Entidades Principais

Modelo	Descrição
User	Representa o usuário autenticado. Possui nome, e-mail, imagem, verificação de e-mail e relações com sessões, contas e favoritos.
Session	Representa uma sessão ativa ou expirada, contendo token, data de expiração, IP, user agent e relação com usuário.
Account	Representa a conta vinculada ao provedor de autenticação, como Google. Armazena tokens e metadados da conta.
Verification	Representa registros de verificação usados pela autenticação.
FavoritePlace	Representa um local salvo como favorito por um usuário. Contém nome, categoria, descrição, avaliação, horário e coordenadas.

10.3 Modelo FavoritePlace

O modelo `FavoritePlace` é central para a funcionalidade de favoritos:

```
1 model FavoritePlace {
2   id String @id @default(cuid())
3
4   userId String
5   user User @relation(fields: [userId], references: [id], onDelete:
6     Cascade)
7
8   placeId String
9   name String
10  category String
11  description String?
12  rating Float?
13  hours String?
14  latitude Float?
```

```

14 longitude    Float?
15
16 createdAt DateTime @default(now())
17 updatedAt DateTime @updatedAt
18
19 @@unique([userId, placeId])
20 @@index([userId])
21 @@map("favorite_place")
22 }

```

Listing 10.1 – Modelo FavoritePlace no Prisma

O índice único `@@unique([userId, placeId])` impede que o mesmo usuário favorite o mesmo local mais de uma vez. Isso é importante para consistência dos dados.

10.4 Relacionamentos

- Um User pode ter várias Session.
- Um User pode ter várias Account.
- Um User pode ter vários FavoritePlace.
- Se um usuário for removido, suas sessões, contas e favoritos são removidos em cascata.

10.5 Diagrama Entidade-Relacionamento Simplificado

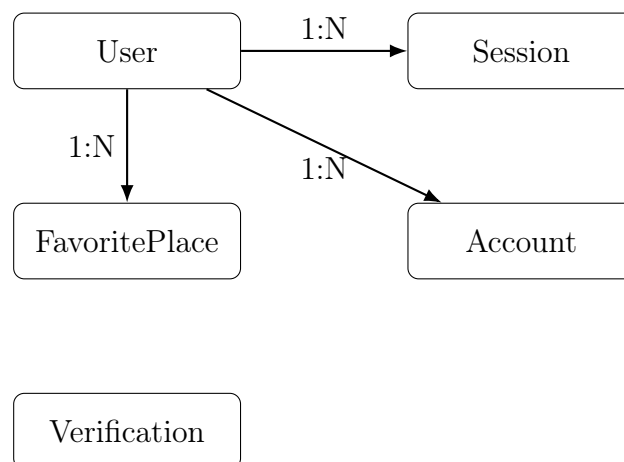


Figura 2 – Relacionamentos principais do banco de dados

11 Fluxo de Busca por CEP

11.1 Visão Geral do Fluxo

A busca por CEP é a principal funcionalidade do sistema. Ela recebe um CEP, valida a entrada, consulta endereço, transforma a cidade em coordenadas, busca o clima e encontra locais próximos. O backend orquestra tudo em uma única chamada para o frontend.

11.2 Fluxo Ponta a Ponta

1. Usuário informa um CEP na tela inicial.
2. Frontend chama `GET /api/search/:cep` usando Axios.
3. Backend verifica se o usuário está autenticado.
4. Controller valida a existência do CEP na rota.
5. Service remove caracteres não numéricos e valida 8 dígitos.
6. Backend chama a API ViaCEP.
7. Backend obtém cidade e UF do endereço retornado.
8. Backend chama Open-Meteo Geocoding para obter latitude e longitude.
9. Backend chama Open-Meteo Forecast para obter clima.
10. Backend chama Overpass API para obter locais próximos.
11. Backend monta o objeto final com endereço, clima e locais.
12. Frontend exibe resultado ou mensagem de erro.

11.3 Orquestração no Service

A função `searchRegion` é responsável por unir todas as etapas:

```
1 export async function searchRegion(cep: string): Promise<SearchResult> {  
2   const address = await getAddressByCep(cep);  
3   const coordinates = await getCoordinates(address.localidade);  
4  
5   const [weather, places] = await Promise.all([  
6     getWeather(coordinates.latitude, coordinates.longitude),
```

```

7     getNearbyPlaces(coordinates.latitude, coordinates.longitude).catch
      (() => []),
8   ]);
9
10   return { address, weather, places };
11 }

```

Listing 11.1 – Orquestração da busca no backend

Um ponto positivo é o uso de `Promise.all` para buscar clima e locais em paralelo. Outro ponto positivo é que, se a busca de locais falhar, o sistema ainda retorna endereço e clima, usando lista vazia para locais.

11.4 Validação do CEP

A validação remove caracteres não numéricos e exige exatamente 8 dígitos:

```

1 const cleanCep = cep.replace(/\D/g, "");
2
3 if (cleanCep.length !== 8 || !/^\d{8}$/.test(cleanCep)) {
4   throw new Error("CEP inválido. Informe 8 dígitos numéricos.");
5 }

```

Listing 11.2 – Validação do CEP

Essa abordagem permite aceitar entradas como 57000-000, pois o hífen é removido antes da validação.

11.5 Integração com ViaCEP

A função `getAddressByCep` consulta:

<https://viacep.com.br/ws/{cep}/json/>

O retorno é convertido para um objeto contendo CEP, logradouro, bairro, localidade e UF. Se o CEP não existir ou os dados vierem incompletos, o service lança erro.

11.6 Integração com Open-Meteo Geocoding

Após obter a cidade no ViaCEP, o backend consulta:

<https://geocoding-api.open-meteo.com/v1/search>

A API retorna latitude e longitude. O backend valida se os valores são numéricos antes de prosseguir.

11.7 Integração com Open-Meteo Forecast

Com latitude e longitude, o backend consulta clima atual e temperaturas diárias:

```
1 const params = new URLSearchParams({
2   latitude: String(latitude),
3   longitude: String(longitude),
4   current: "temperature_2m,weather_code",
5   daily: "temperature_2m_max,temperature_2m_min",
6   timezone: "auto",
7 });
```

Listing 11.3 – Parâmetros de clima

O código meteorológico é traduzido para textos em português, como Céu limpo, Nublado, Chuva leve e Pancadas de chuva.

11.8 Integração com Overpass API

A função `getNearbyPlaces` monta uma query Overpass para buscar estabelecimentos em um raio padrão de 3000 metros. Entre as categorias pesquisadas estão restaurantes, cafês, lanchonetes, farmácias, hospitais, bancos, mercados, lojas, atrações turísticas, parques, academias, escolas, universidades, postos de combustível e outros.

O resultado é filtrado para locais com nome e convertido para o formato:

```
1 {
2   id: place.id,
3   name: place.tags?.name || "Nome n o informado",
4   category: translateCategory(place),
5   description: getDescription(place),
6   latitude: place.lat || place.center?.lat || latitude,
7   longitude: place.lon || place.center?.lon || longitude,
8   rating: null,
9   hours: place.tags?.opening_hours || "Hor rio n o informado",
10 }
```

Listing 11.4 – Formato de local retornado ao frontend

A função retorna no máximo 12 locais, evitando sobrecarregar a interface.

12 Sistema de Favoritos

12.1 Visão Geral

O sistema de favoritos permite que o usuário autenticado salve locais de interesse. A funcionalidade envolve frontend, backend, middleware de autenticação, controller, service e banco de dados.

12.2 Rotas de Favoritos

As rotas estão protegidas por `requireAuth`:

```
1 routes.use(requireAuth);
2 routes.get("/", listFavoritesController);
3 routes.post("/", createFavoriteController);
4 routes.delete("/:placeId", deleteFavoriteController);
```

Listing 12.1 – Rotas de favoritos

12.3 Adicionar Favorito

Para criar um favorito, o backend exige `placeId`, `name` e `category`. Os demais campos são opcionais. O service usa `upsert`, que atualiza o registro caso ele já exista ou cria um novo caso não exista.

```
1 return prisma.favoritePlace.upsert({
2   where: {
3     userId_placeId: {
4       userId: data.userId,
5       placeId: data.placeId,
6     },
7   },
8   update: {
9     name: data.name,
10    category: data.category,
11    description: data.description,
12    rating: data.rating,
13    hours: data.hours,
14    latitude: data.latitude,
15    longitude: data.longitude,
16  },
17  create: data,
18 });
```

Listing 12.2 – Upsert de favorito

Essa estratégia é adequada porque evita duplicidade e mantém os dados atualizados.

12.4 Listar Favoritos

A listagem filtra por `userId` e ordena por data de criação decrescente, mostrando primeiro os favoritos mais recentes.

12.5 Remover Favorito

A remoção usa a chave composta `userId_placeId`, garantindo que um usuário só remova seus próprios favoritos.

12.6 FavoritesProvider no Frontend

No frontend, o `FavoritesProvider` mantém os favoritos em estado global. Ele expõe:

- `favorites`: lista atual de favoritos;
- `isLoading`: estado de carregamento;
- `add`: adiciona favorito;
- `remove`: remove favorito;
- `isFavorite`: verifica se um local já está salvo;
- `refresh`: recarrega a lista.

Esse provider evita repetição de lógica nas páginas e componentes.

13 Análise do Frontend

13.1 Visão Geral

O frontend é uma aplicação React criada com Vite e TypeScript. Ele consome a API backend por Axios, usa BetterAuth para sessão, React Router para navegação e Tailwind CSS para estilização. A proposta visual é moderna, responsiva e centrada em telas principais simples.

13.2 Scripts do Frontend

```
1 {  
2   "dev": "vite",  
3   "build": "tsc -b && vite build",  
4   "lint": "eslint .",  
5   "preview": "vite preview"  
6 }
```

Listing 13.1 – Scripts principais do frontend

13.3 Rotas da Aplicação

O arquivo `App.tsx` define as páginas principais:

```
1 <Routes>  
2   <Route path="/" element={<InicioPage />} />  
3   <Route path="/login" element={<LoginPage />} />  
4   <Route path="/favoritos" element={<FavoritosPage />} />  
5   <Route path="/sobre" element={<SobrePage />} />  
6 </Routes>  
7 {pathname !== '/login' && <NavigationMobile />}  
8 <PlaceDetailModal />
```

Listing 13.2 – Rotas principais do frontend

A navegação mobile aparece em todas as telas, exceto na tela de login. O modal de detalhes fica global, podendo ser aberto a partir de diferentes partes da interface.

13.4 Cliente HTTP

O arquivo `src/lib/axios.ts` cria uma instância Axios com `baseURL` definida por `VITE_API_URL`. O uso de `withCredentials: true` é essencial para enviar cookies de

sessão ao backend.

```
1 export const api = axios.create({
2   baseURL: import.meta.env.VITE_API_URL,
3   withCredentials: true,
4   headers: {
5     "Content-Type": "application/json",
6   },
7 });
```

Listing 13.3 – Cliente HTTP com credenciais

13.5 Hook useSearch

O hook `useSearch` controla a busca por CEP e os estados de tela:

- `idle`: estado inicial;
- `loading`: busca em andamento;
- `success`: busca concluída com sucesso;
- `error`: falha na busca.

Ele chama `/api/search/:cep`, armazena os dados retornados e trata mensagens de erro vindas do backend.

13.6 Providers

A aplicação utiliza providers para organizar estado global:

Provider	Responsabilidade
AuthProvider	Mantém dados do usuário autenticado, estado de carregamento, login com Google e logout.
FavoritesProvider	Mantém lista de favoritos, adiciona, remove, verifica e recarrega favoritos.

13.7 Páginas Principais

13.7.1 Tela de Login

A tela de login é a porta de entrada para autenticação. Ela deve permitir que o usuário inicie login com Google. Após autenticação, o usuário é redirecionado para a página inicial.

13.7.2 Tela Inicial

A tela inicial é onde ocorre a busca por CEP. Ela deve conter campo de entrada, botão de pesquisa, estados visuais de carregamento, erro e sucesso, além da exibição dos cards de endereço, clima e locais próximos.

13.7.3 Tela de Favoritos

A tela de favoritos lista os locais salvos pelo usuário. Ela depende do `FavoritesProvider` e da API `/api/favorites`. A exibição deve permitir visualizar informações do local e remover favoritos.

13.7.4 Tela Sobre

A tela sobre apresenta informações institucionais do projeto, integrantes e objetivo acadêmico.

14 Fluxos End-to-End

14.1 Fluxo 1: Login do Usuário

Etapa	Descrição
1	Usuário acessa <code>/login</code> .
2	Frontend chama <code>authClient.signIn.social</code> .
3	BetterAuth redireciona para Google.
4	Google autentica o usuário e retorna ao backend.
5	BetterAuth grava ou atualiza dados de usuário, conta e sessão no PostgreSQL.
6	Backend define cookie seguro de sessão.
7	Usuário retorna para o frontend.
8	<code>AuthProvider</code> detecta sessão ativa.

14.2 Fluxo 2: Busca por CEP

Etapa	Descrição
1	Usuário informa CEP na página inicial.
2	<code>useSearch</code> altera estado para <code>loading</code> .
3	Frontend chama <code>GET /api/search/:cep</code> .
4	Backend valida sessão com <code>requireAuth</code> .
5	Controller chama <code>searchRegion</code> .
6	Service consulta ViaCEP.
7	Service consulta Open-Meteo Geocoding.
8	Service consulta clima e locais próximos.
9	Backend retorna <code>address</code> , <code>weather</code> e <code>places</code> .
10	Frontend altera estado para <code>success</code> e renderiza os dados.

14.3 Fluxo 3: Adicionar Favorito

Etapa	Descrição
1	Usuário seleciona um local para favoritar.
2	Frontend chama <code>POST /api/favorites</code> .

3	Backend valida sessão.
4	Controller valida dados obrigatórios.
5	Service executa <code>upsert</code> no Prisma.
6	PostgreSQL grava ou atualiza o favorito.
7	Backend retorna favorito criado.
8	Frontend atualiza estado local de favoritos.

14.4 Fluxo 4: Remover Favorito

Etapas	Descrição
1	Usuário aciona remoção de favorito.
2	Frontend chama <code>DELETE /api/favorites/:placeId</code> .
3	Backend valida sessão.
4	Service remove registro usando <code>userId</code> e <code>placeId</code> .
5	Backend retorna status 204.
6	Frontend remove item do estado local.

15 APIs Externas

15.1 ViaCEP

A ViaCEP é utilizada para transformar o CEP em dados de endereço. Ela retorna campos como CEP, logradouro, bairro, localidade e UF. No projeto, essa é a primeira API chamada no fluxo de busca.

15.2 Open-Meteo Geocoding

A API de geocodificação da Open-Meteo transforma o nome da cidade em coordenadas geográficas. Essa etapa é necessária porque as APIs de clima e locais próximos trabalham com latitude e longitude.

15.3 Open-Meteo Forecast

A Open-Meteo Forecast retorna dados climáticos atuais e diários. O projeto usa temperatura atual, código do clima, temperatura máxima e mínima.

15.4 Overpass API / OpenStreetMap

A Overpass API permite consultar dados do OpenStreetMap. O projeto usa essa API para buscar estabelecimentos próximos dentro de um raio de 3000 metros. A query contempla categorias variadas de comércio, saúde, educação, turismo, lazer e serviços.

15.5 Análise das Integrações

API	Entrada	Saída Utilizada
ViaCEP	CEP	Endereço, bairro, cidade e UF.
Open-Meteo Geocoding	Nome da cidade	Latitude e longitude.
Open-Meteo Forecast	Latitude e longitude	Temperatura atual, condição, mínima e máxima.
Overpass API	Latitude, longitude e raio	Lista de estabelecimentos próximos.

16 Deploy e Configuração de Ambiente

16.1 Variáveis do Backend

Variável	Finalidade
DATABASE_URL	URL de conexão com PostgreSQL.
BETTER_AUTH_SECRET	Segredo usado pela autenticação.
BETTER_AUTH_URL	URL pública do backend para callbacks e rotas de autenticação.
FRONTEND_URL	URL pública do frontend autorizada no CORS e no BetterAuth.
GOOGLE_CLIENT_ID	Identificador OAuth do Google.
GOOGLE_CLIENT_SECRET	Segredo OAuth do Google.
PORT	Porta usada pelo servidor, quando aplicável.

16.2 Variáveis do Frontend

Variável	Finalidade
VITE_API_URL	URL base usada pelo Axios e pelo BetterAuth Client para falar com o backend.
VITE_FRONTEND_URL	URL do próprio frontend usada como callback após login.

16.3 Pipeline de Build

16.3.1 Backend

O backend executa:

```
1 npm install
2 npm run build
3 npm start
```

O build executa `prisma generate` antes de compilar TypeScript. Isso garante que o Prisma Client seja criado antes da execução.

16.3.2 Frontend

O frontend executa:

```
1 npm install
2 npm run build
3 npm run preview
```

Em produção, o build do Vite gera arquivos estáticos que podem ser servidos por Vercel, Nginx ou outro serviço de hospedagem.

17 Testes Recomendados

17.1 Testes Funcionais

Caso	Descrição
TF01	Acessar página de login e iniciar autenticação com Google.
TF02	Validar redirecionamento após login.
TF03	Buscar CEP válido e verificar endereço, clima e locais.
TF04	Buscar CEP inválido e verificar mensagem de erro.
TF05	Buscar CEP inexistente e verificar tratamento do ViaCEP.
TF06	Favoritar local e verificar persistência no banco.
TF07	Listar favoritos após recarregar a página.
TF08	Remover favorito e verificar atualização visual.
TF09	Tentar acessar busca sem sessão e esperar status 401.
TF10	Testar logout e invalidação da sessão.

17.2 Testes de Integração

- Testar integração frontend-backend com cookies;
- Testar backend com banco PostgreSQL real;
- Testar BetterAuth com Google OAuth em ambiente de produção;
- Testar APIs externas com CEPs de cidades diferentes;
- Testar comportamento quando Overpass falhar.

17.3 Testes de Usabilidade

- Verificar clareza do campo de CEP;
- Avaliar mensagens de erro;
- Verificar responsividade em telas pequenas;
- Testar navegação inferior mobile;
- Avaliar facilidade para favoritar e remover locais.

18 Pontos Fortes, Limitações e Melhorias

18.1 Pontos Fortes

- Separação clara entre frontend e backend;
- Uso de TypeScript nos dois lados da aplicação;
- Autenticação social moderna;
- Uso de Prisma com PostgreSQL;
- Boa divisão entre controllers, services, routes e libs no backend;
- Uso de providers e hooks no frontend;
- Integração com múltiplas APIs externas;
- Tratamento de erro para CEP inválido e dados incompletos;
- Uso de `upsert` para evitar duplicidade em favoritos;
- Busca paralela de clima e locais para melhorar desempenho.

18.2 Limitações Identificadas

- Dependência forte de APIs externas sem cache;
- Sem testes automatizados identificados na estrutura principal;
- Sem rate limiting nas rotas de busca;
- Tratamento de erro ainda genérico em algumas partes;
- Ratings dos locais retornam `null`, pois o OpenStreetMap nem sempre fornece avaliações;
- Favoritos são carregados no provider mesmo que o usuário não esteja autenticado, podendo gerar requisições 401 silenciosas;
- A busca de coordenadas usa apenas o nome da cidade, o que pode gerar ambiguidade em cidades com nomes repetidos;
- A query Overpass é grande e pode sofrer timeout em regiões densas.

18.3 Melhorias Futuras

1. Implementar cache de resultados por CEP;
2. Adicionar testes unitários e de integração;
3. Implementar rate limiting no backend;
4. Melhorar busca de coordenadas usando cidade, UF e país;
5. Adicionar filtros por categoria de local;
6. Criar paginação ou carregamento progressivo de estabelecimentos;
7. Adicionar mapa interativo com marcadores;
8. Adicionar logs estruturados;
9. Melhorar mensagens de erro no frontend;
10. Adicionar proteção visual de rotas no frontend;
11. Implementar modo offline parcial com últimos resultados;
12. Criar documentação Swagger/OpenAPI para o backend;
13. Adicionar testes E2E com Playwright ou Cypress;
14. Adicionar pipeline CI/CD com GitHub Actions.

19 Dificuldades Encontradas

19.1 Autenticação em Ambientes Diferentes

Uma das principais dificuldades em aplicações com frontend e backend separados é a configuração correta de autenticação, cookies e redirecionamento. Como o BetterAuth utiliza cookies e callbacks, é necessário alinhar `BETTER_AUTH_URL`, `FRONTEND_URL`, `VITE_API_URL`, credenciais do Google e domínios autorizados.

19.2 CORS e Cookies

Quando o frontend e o backend estão em domínios diferentes, o CORS precisa permitir credenciais. O frontend também precisa usar `withCredentials: true`. Se qualquer uma dessas partes estiver incorreta, a sessão pode ser criada no backend, mas não reconhecida no frontend.

19.3 APIs Externas

O sistema depende de serviços externos. ViaCEP, Open-Meteo e Overpass podem ter indisponibilidade, lentidão ou respostas incompletas. Por isso, o backend precisa tratar falhas e manter a experiência do usuário consistente.

19.4 Ambiguidade Geográfica

O fluxo usa a cidade retornada pelo CEP para buscar coordenadas. Em alguns casos, cidades com nomes iguais ou semelhantes podem causar retorno impreciso. Uma melhoria seria usar também UF, país e talvez coordenadas de fontes mais específicas.

20 Resultados e Discussão

A análise demonstra que o Guia Local Inteligente é uma aplicação bem estruturada para fins acadêmicos e com características reais de mercado. O projeto explora separação entre cliente e servidor, autenticação com provedor externo, persistência relacional, consumo de múltiplas APIs e interface responsiva.

O backend concentra corretamente as integrações sensíveis e regras de negócio. Isso evita que o frontend precise chamar diretamente todas as APIs externas e torna mais fácil aplicar cache, logs, rate limiting e validações no futuro. O frontend, por sua vez, mantém uma estrutura componentizada, com hooks e providers para separar responsabilidades.

O sistema de favoritos está bem modelado, pois associa cada local a um usuário e usa uma chave única composta para evitar duplicidade. A autenticação com Google torna a experiência mais simples para o usuário e reduz a necessidade de armazenar senhas próprias.

Como ponto de evolução, a aplicação pode se beneficiar de testes automatizados, cache, documentação OpenAPI, melhoria na precisão de geocodificação, proteção de rotas no frontend e uso de mapas interativos. Mesmo assim, a base atual já cobre os principais conceitos esperados em uma aplicação full stack moderna.

21 Conclusão

O Guia Local Inteligente apresenta uma arquitetura consistente, dividida entre frontend React/Vite e backend Express/TypeScript. A aplicação utiliza PostgreSQL e Prisma para persistência, BetterAuth para autenticação social com Google e APIs externas para enriquecer a busca regional. O fluxo principal, baseado em CEP, passa por várias etapas: validação, consulta de endereço, geocodificação, clima, locais próximos e exibição no frontend.

A análise end-to-end mostra que o projeto atende ao objetivo de centralizar informações locais de forma prática e interativa. Além disso, demonstra domínio de conceitos relevantes: API REST, autenticação, banco relacional, componentização, providers, hooks, CORS, variáveis de ambiente e integração entre serviços.

Portanto, o projeto é adequado como entrega acadêmica e possui potencial de evolução para um produto mais robusto, com mapas, filtros, cache, testes automatizados e melhorias de segurança e observabilidade.

22 Trabalhos Futuros

- Implementar mapa interativo com marcadores dos locais próximos;
- Criar filtros por categoria, distância e horário de funcionamento;
- Adicionar cache por CEP e por coordenadas;
- Criar documentação Swagger/OpenAPI;
- Adicionar testes automatizados no frontend e backend;
- Implementar CI/CD com GitHub Actions;
- Melhorar proteção de rotas no frontend;
- Adicionar tela de perfil do usuário;
- Permitir avaliação pessoal dos locais favoritos;
- Criar histórico de buscas recentes;
- Melhorar acessibilidade da interface;
- Adicionar internacionalização e suporte a múltiplos idiomas.

23 Repositórios do Projeto

- Backend: https://github.com/Ruan-nascimento/Guia_Local_Inteligente-APIREST
- Frontend: https://github.com/Ruan-nascimento/Guia_Local_Inteligente-Client
- Aplicação publicada: <https://euruancarlos.com/gli>

Referências

- [1] REACT. *Documentation*. Disponível em: <https://react.dev>. Acesso em: 08 jun. 2026.
- [2] VITE. *Guide*. Disponível em: <https://vite.dev>. Acesso em: 08 jun. 2026.
- [3] TYPESCRIPT. *Documentation*. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 08 jun. 2026.
- [4] EXPRESS. *Express Documentation*. Disponível em: <https://expressjs.com>. Acesso em: 08 jun. 2026.
- [5] PRISMA. *Prisma Documentation*. Disponível em: <https://www.prisma.io/docs>. Acesso em: 08 jun. 2026.
- [6] POSTGRESQL. *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 08 jun. 2026.
- [7] BETTERAUTH. *Documentation*. Disponível em: <https://www.better-auth.com/docs>. Acesso em: 08 jun. 2026.
- [8] VIACEP. *Webservice de CEP*. Disponível em: <https://viacep.com.br>. Acesso em: 08 jun. 2026.
- [9] OPEN-METEO. *Weather Forecast API*. Disponível em: <https://open-meteo.com>. Acesso em: 08 jun. 2026.
- [10] OVERPASS API. *OpenStreetMap Data API*. Disponível em: <https://overpass-api.de>. Acesso em: 08 jun. 2026.
- [11] NASCIMENTO, Ruan. *Guia Local Inteligente ApiREST*. GitHub. Disponível em: https://github.com/Ruan-nascimento/Guia_Local_Inteligente-ApiREST. Acesso em: 08 jun. 2026.
- [12] NASCIMENTO, Ruan. *Guia Local Inteligente Client*. GitHub. Disponível em: https://github.com/Ruan-nascimento/Guia_Local_Inteligente-Client. Acesso em: 08 jun. 2026.

A Apêndice A – Resumo Técnico para Apresentação

A.1 Resumo do Backend

O backend é responsável por autenticar usuários, proteger rotas, consultar APIs externas, persistir favoritos e retornar dados consolidados para o frontend. Ele utiliza Express, TypeScript, BetterAuth, Prisma e PostgreSQL.

A.2 Resumo do Frontend

O frontend é responsável pela interface, navegação, estado de autenticação, busca por CEP, exibição de resultados e gerenciamento visual de favoritos. Ele utiliza React, Vite, TypeScript, Tailwind CSS, Axios, React Router e BetterAuth Client.

A.3 Resumo do Fluxo Principal

O usuário faz login com Google, pesquisa um CEP, o backend consulta ViaCEP, OpenMeteo e Overpass, e o frontend exibe endereço, clima e locais próximos. O usuário pode favoritar locais, que são salvos no PostgreSQL vinculados à sua conta.

B Apêndice B – Checklist de Defesa Oral

- Explicar por que o projeto é dividido em frontend e backend;
- Mostrar o papel do React e do Vite;
- Mostrar o papel do Express e das rotas REST;
- Explicar autenticação com Google e BetterAuth;
- Explicar o uso do PostgreSQL e Prisma;
- Explicar o fluxo CEP → endereço → coordenadas → clima → locais;
- Explicar o sistema de favoritos;
- Citar APIs externas utilizadas;
- Apontar limitações e melhorias futuras.